

Report on the Design and Implementation of the Image Processing Tool WM145-24 Software Development and Security

u5602780

1 Introduction

This report discusses the design, implementation, and testing of the image processing tool. It includes a detailed system flowchart, implemented features, debugging processes, and black-box testing results, demonstrating the functionality of the tool and its resilience against vulnerabilities.

2 System Design and Flowchart

2.1 Cryptographic Signature Data Flow

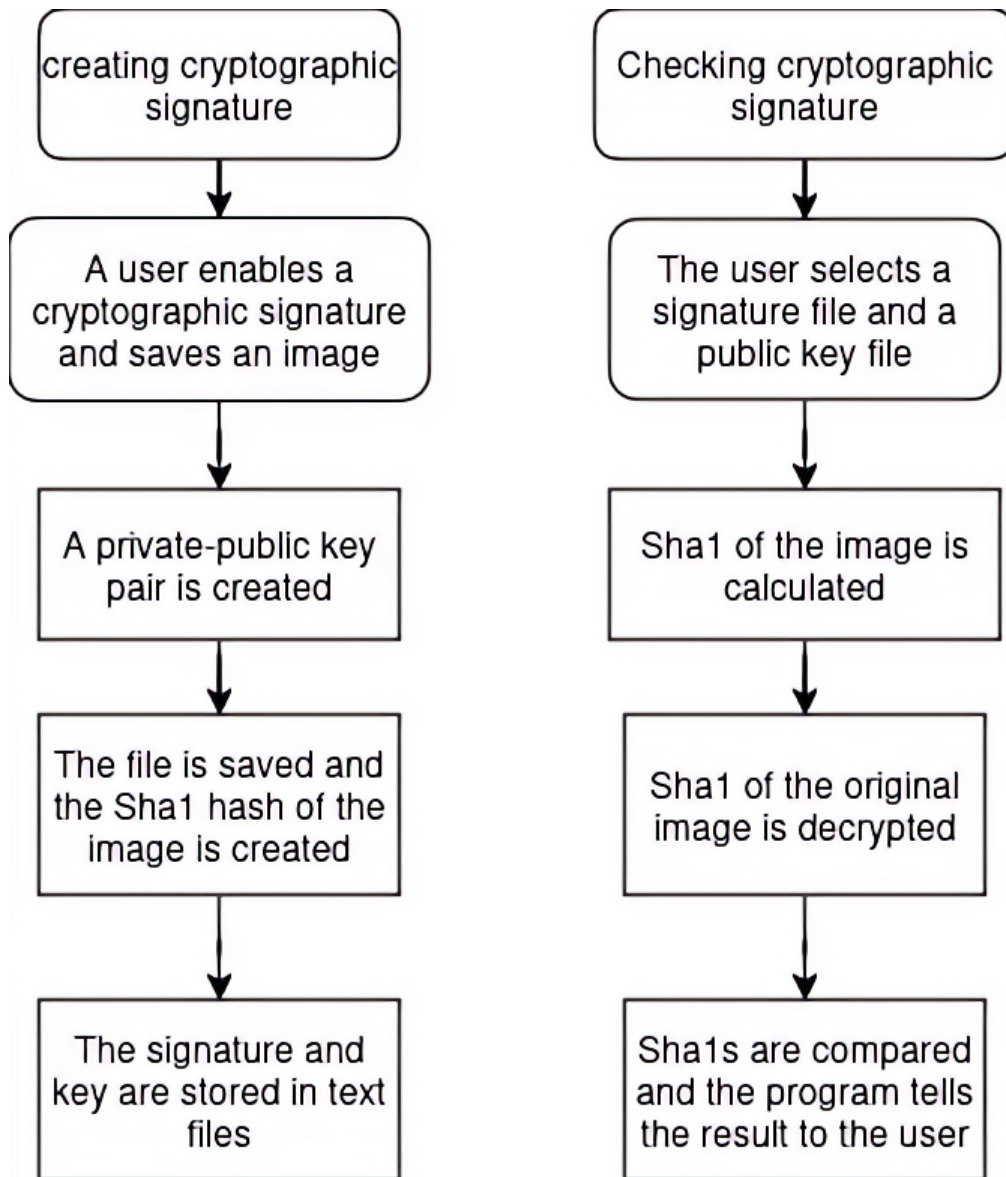


Figure 1: Cryptographic signature in the Image Processing Tool

2.2 Image Data Flow

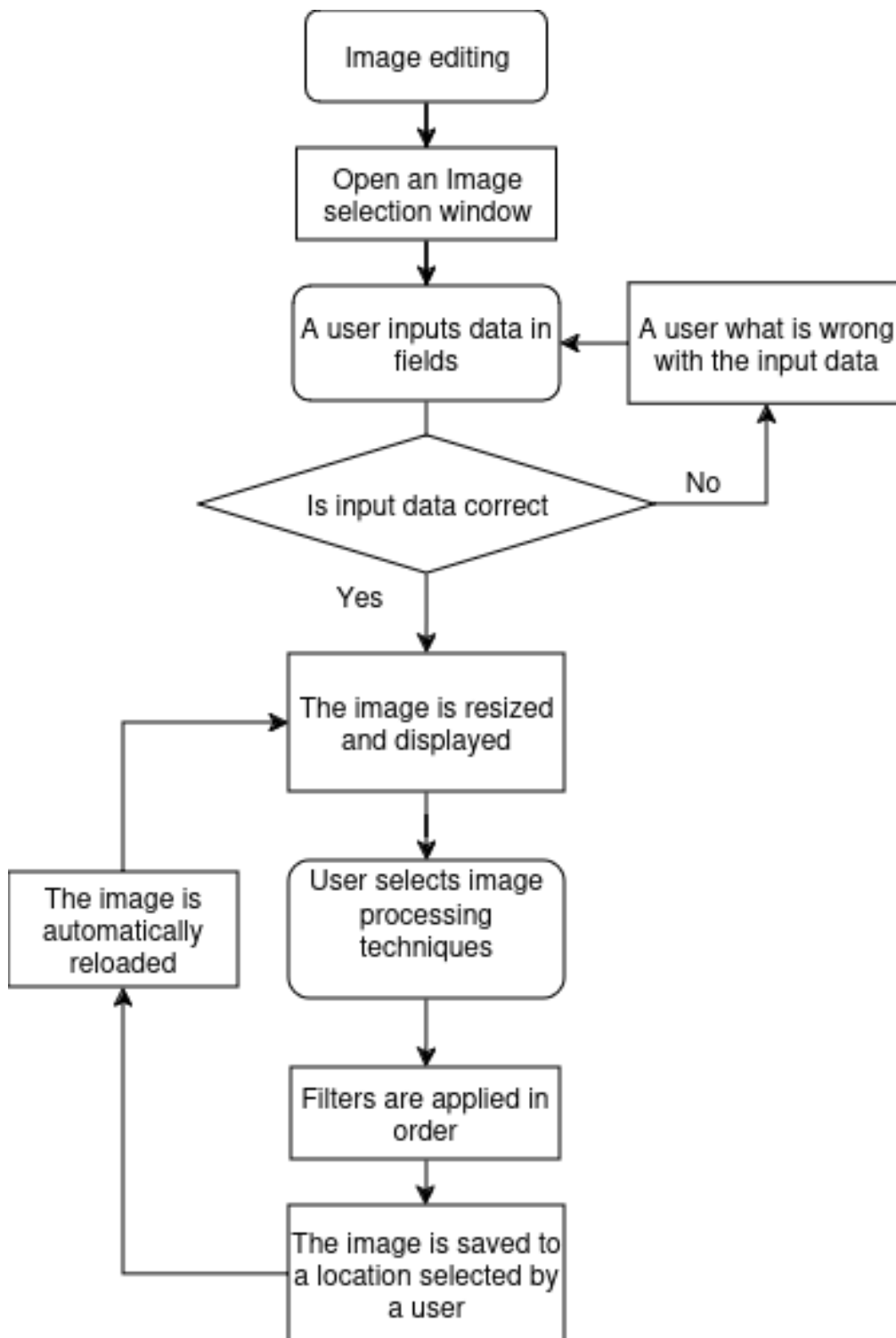


Figure 2: Data flowchart for the Image Processing Tool

3 Implemented Features and Functionalities

3.1 Features

- **File Upload:** This software supports multiple image file formats (JPEG, PNG).
- **Real-time Preview:** Displayed image transforms dynamically and scales to window size.
- **Output Management:** Allows users to save processed images in the desired formats.
- **Processing Options:** Supports edge detection, image inversion, and image blurring. Filters are applied in the order selected by the user.
- **Metadata Management:** Users need to provide details such as the name of the photo, file path, date the photo was taken, date of submission, photographer, and image description. The description must be between 0 and 250 words. After saving, this metadata is stored in the image's metadata and automatically filled when the image is loaded again.
- **Cryptographic Signature:** Allows users to generate and verify cryptographic signatures for any image. When a cryptographic signature is enabled, the program creates two text files: a `.pem` file containing the public key and a `.bin` file containing the encrypted signature using a private-public key pair and an SHA-1 hash script.

3.2 Functionalities

- **Input Validation:** Ensures only valid file types are processed. Checks metadata fields for errors and enforces description length requirements.
- **Error Handling:** Notifies users of any issues that occur in the program.
- **Automatic Metadata Handling:** Automatically populates metadata fields when the image is loaded.
- **Cryptographic Security:** Ensures secure signing and verification of an image using asymmetric encryption.

4 Debugging and Troubleshooting

This section discusses the steps taken to debug and troubleshoot issues encountered during development, along with the measures implemented to prevent possible bugs and errors.

4.1 Image Resizing During Window Resize Events

Bug: When resizing the application window, the displayed image stretched incorrectly or appeared distorted due to the aspect ratio not being maintained.

Debugging Steps:

- Tested the program with images of various resolutions and dimensions.
- Used print statements to log the image's width, height, and calculated aspect ratio during resizing.
- Identified incorrect calculations for scaling the image, particularly for edge cases such as extremely narrow or wide windows.

4.2 GUI Menubar Display Bug

Bug: When launching the program, an additional empty window was displayed alongside the main app. This issue was caused by incorrect implementation of my library wrapper in one of the GUI components used to implement the menu bar.

Debugging Steps:

- The issue was first noticed during the initial launch of the program, when an unexpected empty window appeared.
- To find the reason for the bug, I disabled GUI components one by one.
- The issue was caused by the `CreateMenubar` class, specifically in the initialisation of the `CTkMenuBar` object.
- The issue was fixed by rewriting the entire class and the way different components are called and initialised.

4.3 String Metadata Handling Bug

Bug: Metadata strings were not properly saved in image files. Even after switching to the `PIL.PngImagePlugin.PngInfo()` library, issues persisted because it automatically converted metadata keys to lowercase and replaced spaces with underscores.

Debugging Steps:

- For the first couple of attempts to save metadata, I used a custom dictionary, but it had many issues related to data corruption.
- After switching to the `PIL.PngImagePlugin.PngInfo()` library, metadata was saved more reliably. However, even this library is not ideal and tends to transform keys (e.g., "Image Description" to "image_description").
- This forced me to test by saving and reloading images with different metadata configurations to identify patterns in key transformation.

5 Black-Box Testing Results

The table below summarises the results of black-box testing for key features:

Feature	Test Input	Expected Outcome	Actual Outcome
Applying Filters	Apply filters to an unloaded image	Error message is displayed	Error message is displayed
Cryptographic Signature Corruption	Press all cryptographic buttons without loading image	Error message is displayed	Error message is displayed
File Upload	Upload non-JPG/PNG files	Only JPG/PNG files are displayed	Only JPG/PNG files are displayed
Date Input	Invalid date format (e.g., "32/13/2024")	Input is rejected with an error message	Input is rejected with an error message
Description	Description longer than 250 words	Input is rejected	Input is rejected
Upload Image with Metadata	Load image with pre-saved metadata	Metadata fields are filled automatically	Metadata fields are filled automatically
Save Image	Save processed image	Image is saved without issues	Image is saved without issues
Applying Filters	Apply multiple filters sequentially	Filters are applied in the order selected	Filters are applied in the order selected
Cryptographic Signature Validation	Enable signature and save image	.pem and .bin files are created. Validation works for unmodified images	.pem and .bin files are created. Validation works for unmodified images
Image changes size with changing window size	Load Image and change window size	Size of the image inside the window is adjusted	Size of the image inside the window is adjusted

Table 1: Black-Box Testing Results for Key Features